Tema 5.1: Introducción al Modelo Observador

Pablo Gordillo (pabgordi@ucm.es)

Departamento de Sistemas Informáticos y Computación

Facultad de Informática



(Basado en el material creado por Puri Arenas, Jesús Correas y Samir Genaim)

Ejemplo: Sensores

- Implementamos un programa que **regula la temperatura de una habitación usando sensores que controlan aparatos de aire acondicionado:**
  1. Cuando la temperatura excede de los 25 grados, se enciende el aire acondicionado.
  2. Cuando la temperatura está por debajo de los 25 grados, el aire acondiciponado se apaga.

Primera Implementación

```
public class AirCondition {

  protected boolean isOn;
  protected String id;

  public AirCondition(String id) {
    this.id = id;
    this.isOn = false;
  }

  public void SwitchOnOff(int t) {
    if (t >= 25) this.isOn = true;
    else this.isOn = false;
  }

  @Override
  public String toString(){
    return id + ":" + (this.isOn ? "Encendido" : "Apagado");
  }
  ...
}
```

Primera Implementación

```
public class Sensor {
  private List <AirCondition> airconditions;

  public Sensor() {
    this.airconditions = new ArrayList<AirCondition>();
  }

  public void add(AirCondition l){
    if (!this.airconditions.contains(l))
    this.airconditions.add(l);
  }

  public void run(int t) {
    for (AirCondition la : this.airconditions){
      la.SwitchOnOff(t);
      System.out.println(la);
    }
  }
}
```

Primera Implementación

```
public class Sensor { //EL MODELO
  private List <AirCondition> airconditions;

  public Sensor() {
    this.airconditions = new ArrayList<AirCondition>();
  }

  public void add(AirCondition l){
    if (!this.airconditions.contains(l))
    this.airconditions.add(l);
  }

  public void run(int t) {
    for (AirCondition la : this.airconditions){
      la.SwitchOnOff(t);
      System.out.println(la);
    }
  }
}
```

Primera Implementación

```
public class Controller {
  private Sensor sensor;
  private static final int TemperatureLimit = 41;

  public Controller () {
    this.sensor = new Sensor();
  }

  public void run() throws InterruptedException {
    while (true) {
      int t = (int) (Math.random()*Controller.TemperatureLimit);
      System.out.println ("Temperatura: " + t);
      this.sensor.run(t);
      Thread.sleep(5000);
    }
  }

  public void add(AirCondition a) {
    this.sensor.add(a);
  }
}
```

Primera Implementación

```
public class Main {
  public static void main(String[] args) throws InterruptedException {
    Controller c = new Controller();
    AirCondition a1 = new AirCondition("1");
    AirCondition a2 = new AirCondition("2");
    c.add(a1);
    c.add(a2);
    c.run();
  }
}
```

Primera Implementación

```
public class Main {
  public static void main(String[] args) throws InterruptedException {
    Controller c = new Controller();
    AirCondition a1 = new AirCondition("1");
    AirCondition a2 = new AirCondition("2");
    c.add(a1);
    c.add(a2);
    c.run();
  }
}
```

Y si queremos añadir más aparatos de climatización que funcionen de forma distinta...

Primera Implementación

Y si queremos añadir ahora estufas que funcionan con leña?

```
public class Stove {
  protected int money; // DINERO QUE SUPONE USAR ESTA ESTUFA
  protected String id;

  public Stove(String id){
    this.id = id;
    this.money = 0;
  }

  public void computeMoney(int t) {
    if (t >= 28) this.money = this.money + 300;
    else this.money = this.money + 100;
  }

  @Override
  public String toString(){
    return "Consumo de " + id + ": " + this.money;
  }
}
```

Primera Implementación

```
public class Sensor { //EL MODELO
  private List <AirCondition> airconditions;

  public Sensor() {
    this.airconditions = new ArrayList<AirCondition>();
  }

  public void add(AirCondition l){
    if (!this.airconditions.contains(l))
    this.airconditions.add(l);
  }

  public void run(int t) {
    for (AirCondition la : this.airconditions){
      la.SwitchOnOff(t);
      System.out.println(la);
    }
  }
}
```

Primera Implementación

```
public class Sensor { //EL MODELO
  private List <AirCondition> airconditions;

  public Sensor() {
    this.airconditions = new ArrayList<AirCondition>();
  }

  public void add(AirCondition l){
    if (!this.airconditions.contains(l))
    this.airconditions.add(l);
  }

  public void run(int t) {
    for (AirCondition la : this.airconditions ){
      la.SwitchOnOff(t);
      System.out.println (la);
    }
  }
}
```

Nos vale este modelo?

Primera Implementación

```
public class Sensor { //EL MODELO
  private List <AirCondition> airconditions;

  public Sensor() {
    this.airconditions = new ArrayList<AirCondition>();
  }

  public void add(AirCondition l){
    if (!this.airconditions.contains(l))
    this.airconditions.add(l);
  }

  public void run(int t) {
    for (AirCondition la : this.airconditions ){
      la.SwitchOnOff(t);
      System.out.println (la);
    }
  }
}
```

Nos vale este modelo? NO

Primera Implementación

```
public class Sensor {
  private List<AirCondition> airconditions;
  private List<Stove> stoves;

  public Sensor() {
    this.airconditions = new ArrayList<AirCondition>();
    this.stoves = new ArrayList<Stove>();
  }

  public void add(AirCondition l){
    if (!this.airconditions.contains(l)) this.airconditions.add(l);
  }

  public void add(Stove st){
    if (!this.stoves.contains(st)) this.stoves.add(st);
  }

  public void run(int t) {
    for (AirCondition la : this.airconditions){
      la.SwitchOnOff(t);
      System.out.println(la);}
    for (Stove st : this.stoves){
      st.computeMoney(t);
      System.out.println(st);}
  }
}
```

El método run también cambia

Primera Implementación

```
public class Sensor {
  private List <AirCondition> airconditions;
  private List <Stove> stoves;

  public Sensor() {
    this.airconditions = new ArrayList<AirCondition>();
    this.stoves = new ArrayList<Stove>();
  }

  public void add(AirCondition l){
    if (!this.airconditions.contains(l)) this.airconditions.add(l);
  }

  public void add(Stove st){
    if (!this.stoves.contains(st)) this.stoves.add(st);
  }

  public void run(int t) {
    for (AirCondition la : this.airconditions){
      la.SwitchOnOff(t);
      System.out.println(la);}
    for (Stove st : this.stoves){
      st.computeMoney(t);
      System.out.println(st);}
  }
}
```

El método run también cambia

Modificación de código !!!! Viola un principio muy importante de la OO !!! Las clases tienen que ser abiertas para extensiones y cerradas para modificaciones !!!!

Segunda Implementación: Observadores

1. Vamos a hacer que los aparatos de climatización sean **observadores** del sensor de temperatura
2. De esta forma el sensor de temperatura contendrá una **lista de observadores**, a los que informará en caso de que se produzcan cambios en la temperatura
3. Cada observador (aparato de climatización), cuando reciba una notificación, hará el cambio que considere oportuno
4. A través del controlador se pueden añadir los observadores al sensor de temperatura

Segunda Implementación

Un observador de temperatura necesita saber cuando la temperatura cambia. Por lo tanto ha de implementar la siguiente interfaz:

```
public interface TempObserver {
  public void tempChanged(int t);
}
```

Segunda Implementación

```
public class AirCondition {
  protected boolean isOn;
  private String id;

  public AirCondition(String id){
    this.id = id;
    this.isOn = false;
  }

  private void SwitchOnOff(int t) {
    if (t >= 25) this.isOn = true;
    else this.isOn = false;
  }

  @Override
  public String toString(){
    return "Air Condition " + id + ":" +
    (this.isOn ? "Encendido" : "Apagado");
  }
```

Segunda Implementación

```
public class AirCondition implements TempObserver {
  protected boolean isOn;
  private String id;

  public AirCondition(String id){
    this.id = id;
    this.isOn = false;
  }

  private void SwitchOnOff(int t) {
    if (t >= 25) this.isOn = true;
    else this.isOn = false;
  }

  @Override
  public String toString(){
    return "Air Condition " + id + ":" +
    (this.isOn ? "Encendido" : "Apagado");
  }

  @Override
  public void tempChanged(int t) { this.SwitchOnOff(t); }
  ...
}
```

Segunda Implementación

```
public class Stove implements TempObserver {
  protected int money;
  protected String id;

  public Stove(String id){
    this.id = id;
    this.money = 0;
  }

  private void computeMoney(int t) {
    if (t >= 28) this.money = this.money + 300;
    else this.money = this.money + 100;
  }

  @Override
  public String toString(){
    return "Consumo de " + id + ": " + this.money;
  }

  @Override
  public void tempChanged(int t) { this.computeMoney(t); }
  ...
 }
}
```

Segunda Implementación

```
public class Sensor {
  private List<TempObserver> observers;

  public Sensor() {
    this.observers = new ArrayList<TempObserver>();
  }

  public void run(int t) {
    for (TempObserver ob: observers) {
      ob.tempChanged(t);
      System.out.println(ob);
    }
  }
}
```

Segunda Implementación

```
public class Sensor {
  private List<TempObserver> observers;

  public Sensor() {
    this.observers = new ArrayList<TempObserver>();
  }

  public void run(int t) {
    for (TempObserver ob: observers) {
      ob.tempChanged(t); //todos los observadores son notificados
      System.out.println (ob);
    }
  }
}
```

Segunda Implementación

El sensor de temperatura ahora trabaja sobre una lista de **Observadores de Temperatura. Por lo tanto debe permitir añadir nuevos observadores y también eliminarlos. Para forzar al sensor a esto utilizaremos una nueva interfaz TempObservable**

```
public interface TempObservable {
  public void addObserver(TempObserver ob);
  public void removeObserver(TempObserver ob);
}
```

Segunda Implementación

```
public class Sensor implements TempObservable{
  private List <TempObserver> observers;

  public Sensor() {
    this.observers = new ArrayList<TempObserver>();
  }

  public void run(int t) {
    for (TempObserver ob: observers) {
      ob.tempChanged(t); //todos los observadores son notificados
      System.out.println (ob);
    }
  }

  @Override
  public void addObserver(TempObserver ob) {
    if (!this.observers.contains(ob)) this.observers.add(ob);
  }
}
```

Segunda Implementación

```
public class Controller {
  private Sensor sensor;
  private static final int TemperatureLimit = 41;

  public Controller () {
    this.sensor = new Sensor();
  }

  public void run() throws InterruptedException {
    while (true) {
      int t = (int) (Math.random()*Controller.TemperatureLimit);
      System.out.println ("Temperatura: " + t);
      this.sensor.run(t);
      Thread.sleep(5000);
    }
  }

  public void addObserver(TempObserver ob) {
    this.sensor.addObserver(ob);
  }
  ...
}
```

Segunda Implementación

```
public class Main {

  public static void main(String[] args) throws InterruptedException {
    Controller c = new Controller();

    AirCondition a = new AirCondition("1");

    c.addObserver(a);
    Stove st = new Stove("1");
    c.addObserver(st);
    c.run();
  }
}
```

Segunda Implementación

Ahora si queremos añadir un nuevo sistema de climatización solo tenemos que convertirlo en observador, sin necesidad de tocar ninguna otra clase o el modelo!

```
public class Heater implements TempObserver {
  ...

  @Override
  public void tempChanged(int t) {...}
}

public class Main {
  public static void main(String[] args) throws InterruptedException {
    Controller c = new Controller();
    ...
    Heater h = new Heater("1");
    c.addObserver(h);
    c.run();
  }
}
```

Patrón Observer

El diseño que hemos visto es un patrón muy importante de programación:

El patrón de diseño Observer